



1 Background and Motivation

In a **conventional transceiver design**, coding, modulation, channel estimation and detection are designed as separate blocks. Each block is optimal for the job it is given, but the receiver as a whole is only as good as the assumptions that connect them. Deep learning offers two alternatives: train transmitter and receiver together as an **autoencoder** [1], or replace channel estimation and detection with a **learned receiver** [2].

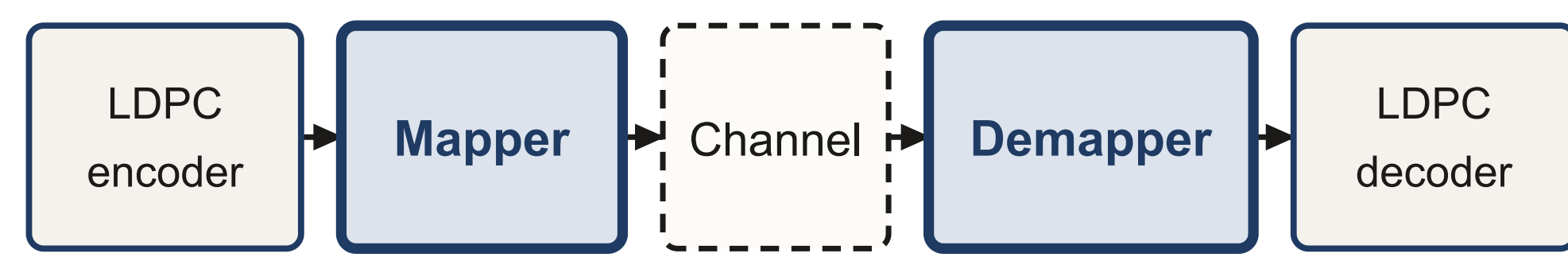
We ask two questions:

- How much is end-to-end learning worth compared with a classical receiver that is already **doing its best**?
- Can the transmitter be trained at all when **no differentiable model of the channel** is available?

2 Objectives and Contributions

- Measure the gain of a learned transceiver against a **matched** classical baseline, on AWGN and on Rayleigh block fading with **pilot-estimated CSI**.
- Train the transmitter with **no channel model**, using reinforcement learning instead of backpropagation through the channel, and measure the cost.
- Account for the pilot overhead and the channel estimation error. Let **R** be the code rate, **L** the number of symbols in one fading block, **P** the number of pilot symbols in that block, **N₀** the noise power spectral density and **$\hat{h}$** the estimated channel coefficient. The effective code rate is then **$R(L-P)/L$** , and after zero-forcing the effective noise is **$N_0(1+1/P)/|\hat{h}|^2$** .
- Identify the cause of each failure by experiment: keep the weights fixed, change one thing at a time, and see which change removes the symptom.

3 System Model

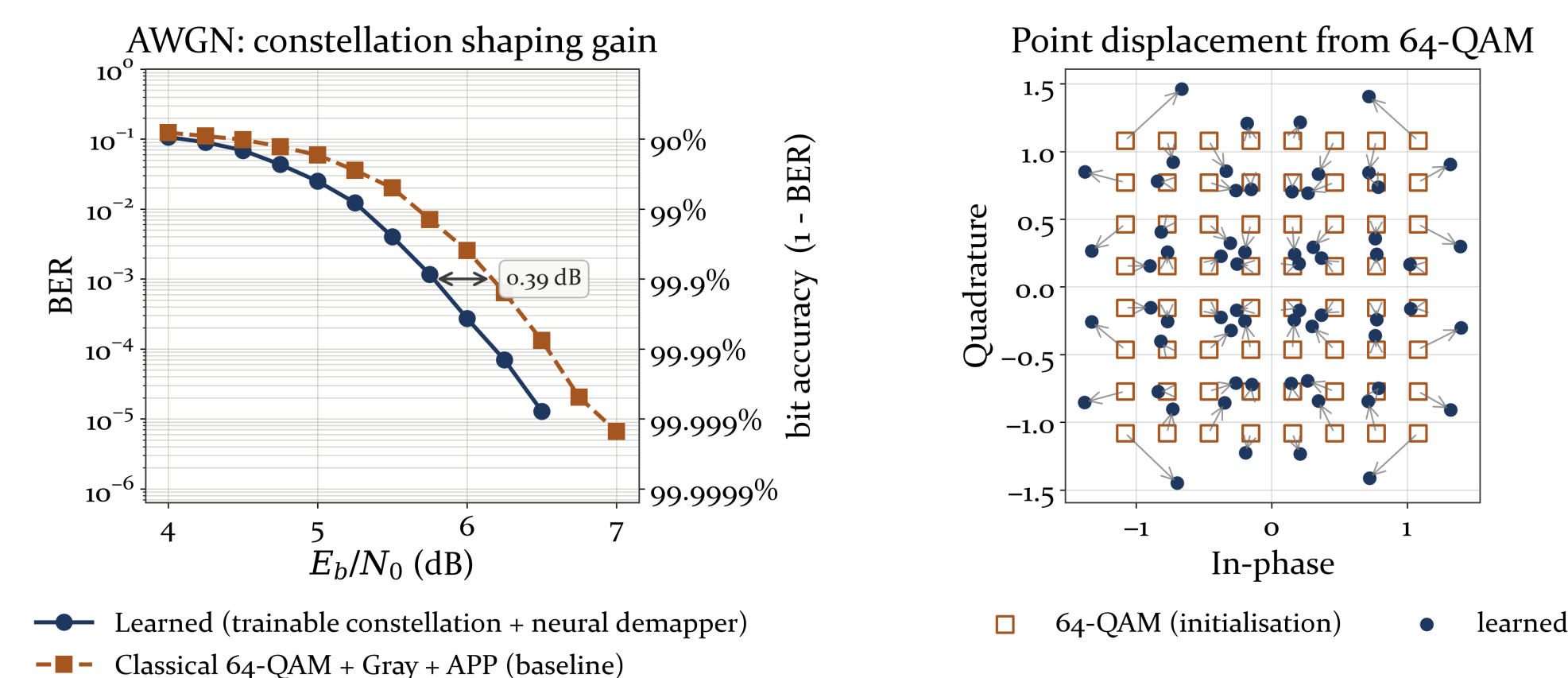


Learned	trainable constellation	neural network
Classical	64-QAM, Gray	APP demapper

Channel: AWGN | Rayleigh block fading, L = 25, P pilot symbols, LS estimate

The two systems differ only in the shaded blocks. They share the rate-1/2 5G LDPC code, the channel model and the random seeds, so any difference in bit error rate comes from the mapper and the demapper.

4 Where the AWGN Gain Comes From



Under AWGN, APP demapping is already optimal, so the improvement cannot come from the receiver. It comes from the transmitter: the learned constellation moves away from the regular 64-QAM lattice and gives more power to the symbols that the demapper confuses most often.

References

- [1] T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer," arXiv:1702.00832, 2017.
- [2] H. Ye, G. Y. Li and B.-H. F. Juang, "Power of Deep Learning for Channel Estimation and Signal Detection in OFDM," arXiv:1708.08514, 2017.
- [3] S. Smeka J. et al., "Accelerating Meta-Learning with the Enhanced Reptile Algorithm for Rapid Adaptation in Neural Networks," SCSE, IEEE, 2025.
- [4] N. C. Luong et al., "Applications of Deep Reinforcement Learning in Communications and Networking: A Survey," IEEE Commun. Surveys Tuts., vol. 21, no. 4, 2019.
- [5] NVIDIA Sionna, autoencoder tutorial notebook, which this work extends.

5 Performance Analysis

Table 1 — E_b/N_0 needed for a given bit error rate (dB, lower is better). 64,000 codewords per point; every entry rests on at least 30 block errors.

System	1e-2	1e-3	1e-4
AWGN, classical 64-QAM + APP	5.67	6.17	6.54
AWGN, learned (channel gradient)	5.30	5.78	6.18
AWGN, learned (RL, no channel model)	5.42	5.92	6.31
Rayleigh, classical 64-QAM + APP	11.26	13.30	14.79
Rayleigh, learned	11.25	13.19	14.94

- Fading costs **5.9 to 8.8 dB**, depending on the target error rate, while the choice of receiver changes the result by less than **0.4 dB**.
- On AWGN the learned system gains **0.37, 0.39 and 0.35 dB** at BER 1e-2, 1e-3 and 1e-4. APP demapping is already optimal here, so the gain comes from **constellation shaping**.
- Leaving the $(1+1/P)$ factor out of the effective noise understates it by **1.76 dB** at $P = 2$.
- Under Rayleigh fading with estimated CSI we measured **no gain** (+0.01, +0.11 and -0.15 dB). The value at 1e-4 changed sign between two runs, so we treat it as noise.

6 Why AWGN Weights Fail on Fading

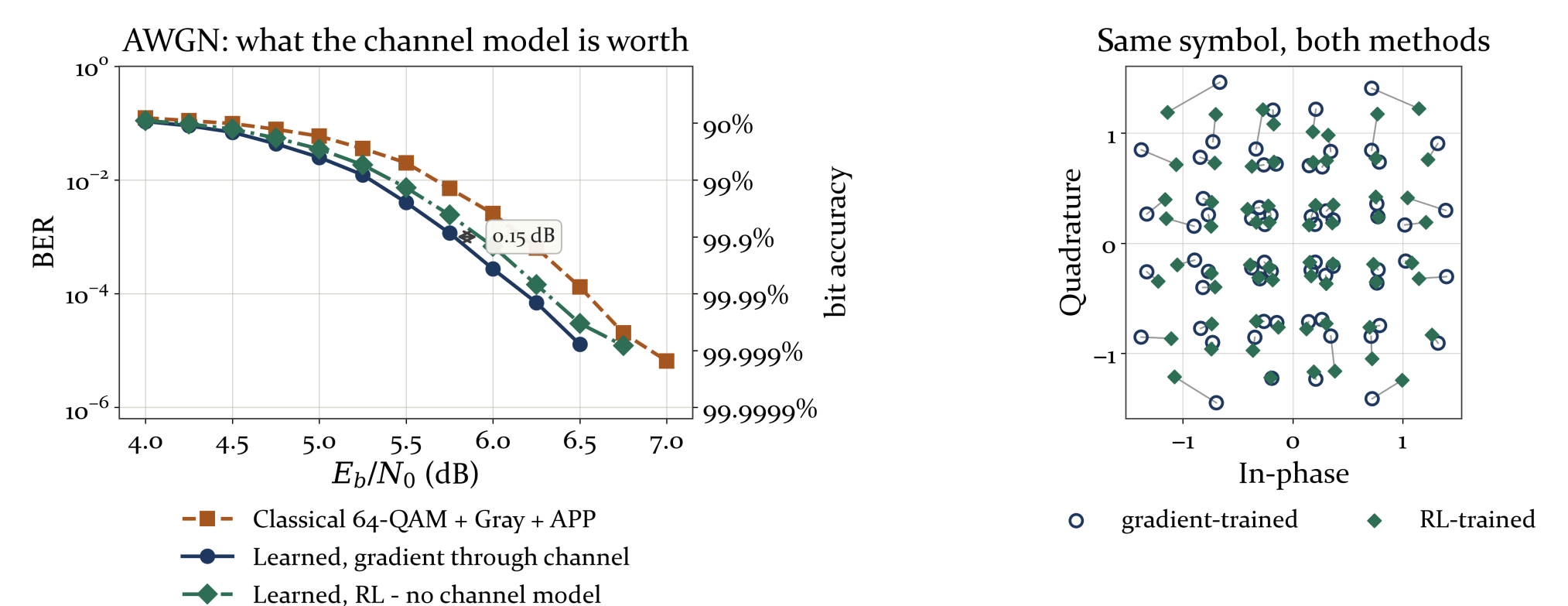
When the AWGN weights are used on a fading channel without retraining, the BER increases as the SNR increases. A real system cannot behave this way, so we kept every weight fixed and changed one thing at a time:

- Clamping the network outputs changed the BER by less than 4%, and the curve still increased.
- Removing the symbols in deep fades made the BER worse above 10 dB.
- Clamping the noise value passed to the demapper into the range it saw during training **restored a decreasing curve**, without changing any weight.

The cause is on the input side: at 18 dB, **89%** of symbols have an effective noise below anything the network saw in training, and only 4.6% are in deep fades. Retraining on the fading channel gives BER **1e-3** at 13.2 dB and **1e-4** at 14.9 dB.

7 Learning Without a Channel Model

All the results above were obtained by backpropagating through a differentiable channel model, which a deployed system does not have. Here the receiver is trained as usual, but its gradient does not pass through the channel. The transmitter perturbs its own symbols, receives one scalar loss value per codeword, and updates the constellation from that value [4]; the channel is sampled rather than differentiated.



- Removing the channel model costs **0.12, 0.15 and 0.12 dB** at BER 1e-2, 1e-3 and 1e-4.
- The reinforcement-learned system is still **0.24 dB** better than classical 64-QAM.
- The two training methods place each symbol at almost the same position, so reinforcement learning converges to a similar constellation.

8 Conclusion

On AWGN, end-to-end learning gives a small but repeatable gain of **0.37 dB**. Under Rayleigh fading with estimated CSI, we measured no gain. The main argument for an AI-native transceiver is therefore not accuracy, but that it can be trained **without a channel model**, and our measurements put the cost of doing so at **0.12 to 0.15 dB**.

9 Next Steps

- Deploy on real hardware.** Embed the trained transceiver in a real device, such as a software-defined radio, and check whether the gain survives outside simulation.
- Meta-learning for fast adaptation** [3]. The section above shows that the receiver fails when its input distribution changes, so adaptation is the problem to solve.
- Extend to OFDM and multipath** [2], where a convolutional demapper can use the correlation between neighbouring subcarriers.
- Replace zero-forcing with MMSE equalisation**, which keeps the effective noise bounded.

Points resting on fewer than 30 block errors are dropped rather than drawn. Seeds are fixed, and every number is measured in the accompanying notebook. Sionna [5] and PyTorch. **Scope:** flat block fading only, with no multipath, Doppler or OFDM.